Verification of Probing Security for Masked Circuits

A Constructive Approach and Theoretical Study

Kağan Gülsüm

July 21, 2026

Sabancı University — M.Sc. Thesis Defense
Supervisor: *Asst. Prof. Süha Orhun Mutluergil*

The story of this talk

1. **Problem:** real devices leak; measuring power or timing can recover secret keys. **Standard fix:** masking; compute on randomized shares, never on secrets.

The story of this talk

1. **Problem:** real devices leak; measuring power or timing can recover secret keys. **Standard fix:** masking; compute on randomized shares, never on secrets.
2. **The question:** was the masking done right? "Right" means no d observed wires can tell two secrets apart.

The story of this talk

1. **Problem:** real devices leak; measuring power or timing can recover secret keys. **Standard fix:** masking; compute on randomized shares, never on secrets.
2. **The question:** was the masking done right? "Right" means no d observed wires can tell two secrets apart.
3. **It is provably hard:** as hard as counting solutions over all input combinations, so no tool can simply compute the answer.

The story of this talk

1. **Problem:** real devices leak; measuring power or timing can recover secret keys. **Standard fix:** masking; compute on randomized shares, never on secrets.
2. **The question:** was the masking done right? "Right" means no d observed wires can tell two secrets apart.
3. **It is provably hard:** as hard as counting solutions over all input combinations, so no tool can simply compute the answer.
4. **Our route:** prove security by re-arranging the randomness until the secrets visibly cancel. Coppula does this in Lean 4, with a rewrite engine, a combinatorial search, and reusable certificates.

The story of this talk

1. **Problem:** real devices leak; measuring power or timing can recover secret keys. **Standard fix:** masking; compute on randomized shares, never on secrets.
2. **The question:** was the masking done right? "Right" means no d observed wires can tell two secrets apart.
3. **It is provably hard:** as hard as counting solutions over all input combinations, so no tool can simply compute the answer.
4. **Our route:** prove security by re-arranging the randomness until the secrets visibly cancel. Coppula does this in Lean 4, with a rewrite engine, a combinatorial search, and reusable certificates.
5. **Result:** soundness proofs, gadgets verified that a comparable tool false-flags, and competitive performance.

Motivation & Model

Side-channel attacks

A cipher can be secure on paper and insecure on the bench.

- A real device leaks through power draw, timing, electromagnetic emanation; all of which vary with the *data* it handles.
- Differential power analysis (Kocher et al. 1999) correlates many power traces against a guess of an intermediate value, and may recover the key.
- In practice: DPA recovered keys from deployed smartcards; timing channels underlie Spectre and Meltdown.

So, the insecurity lies not in the mathematics behind the cipher, but in the way the hardware computes it.

The setting

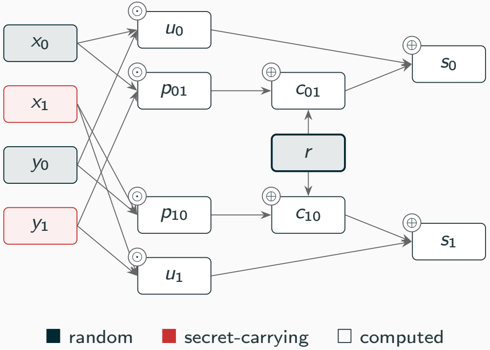
We study **arithmetic circuits** over the two-element field $\mathbb{F}_2$, built from two-input $\oplus$ (XOR) and $\odot$ (AND) gates.

Inputs are partitioned into three classes

- **secrets** $\mathcal{S}$ — the key material we protect,
- **randoms** $\mathcal{R}$ — fresh uniform bits (the masks), drawn together as a *tape* $\rho \in \{0,1\}^{\mathcal{R}}$,
- **publics** $\mathcal{P}$ — known to everyone, held fixed.

Masking, and our running example

Boolean masking (Chari et al. 1999): split each secret into shares, $x = x_0 \oplus x_1$ with x_0 uniformly random, and compute only on shares, never on secrets.



DOM-AND (Groß et al. 2016): a masked AND gate, two shares per input, one fresh mask r.

- outputs re-share $s_0 \oplus s_1 = x \odot y$,
- cross terms p_{01}, p_{10} mix shares of *both* secrets,
- r re-masks them before the outputs.

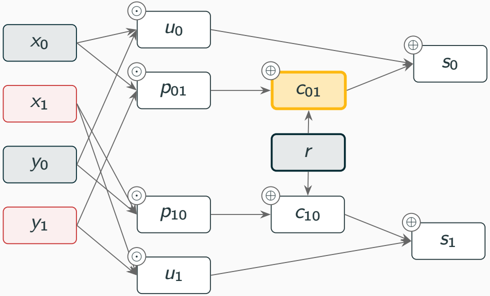
The probing model

Reasoning about noisy traces directly is unwieldy. Luckily we have an abstraction (Ishai et al. 2003):

d-probing security

An adversary picks any d wires $O = \{w_1, \ldots, w_d\}$ and is handed their *exact* values. The circuit is d-probing secure when, for every such choice and every two secret assignments a, b, the observed values are identically distributed over the tape ρ:

$$\mathcal{L}_{\mathsf{a}}(w_1, \ldots, w_d) \; = \; \mathcal{L}_{\mathsf{b}}(w_1, \ldots, w_d).$$

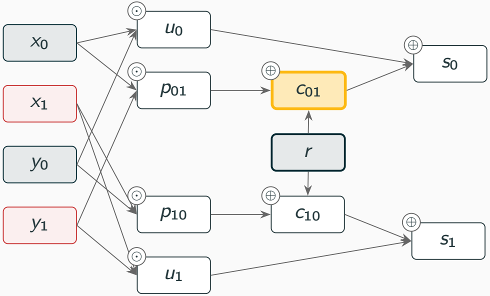


The probing model

Reasoning about noisy traces directly is unwieldy. Luckily we have an abstraction (Ishai et al. 2003):

d-probing security
An adversary picks any d wires $O = \{w_1, \dots, w_d\}$ and is handed their *exact* values. The circuit is d-probing secure when, for every such choice and every two secret assignments a, b, the observed values are identically distributed over the tape ρ:

$$\mathcal{L}_{\mathrm{a}}(w_1, \dots, w_d) = \mathcal{L}_{\mathrm{b}}(w_1, \dots, w_d).$$



- Security becomes a *combinatorial* property, decidable in principle.

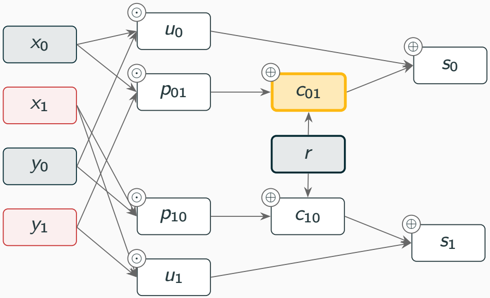
The probing model

Reasoning about noisy traces directly is unwieldy. Luckily we have an abstraction (Ishai et al. 2003):

d-probing security

An adversary picks any d wires $O = \{w_1, \ldots, w_d\}$ and is handed their *exact* values. The circuit is d-probing secure when, for every such choice and every two secret assignments a, b, the observed values are identically distributed over the tape ρ:

$$\mathcal{L}_{\mathrm{a}}(w_1, \ldots, w_d) = \mathcal{L}_{\mathrm{b}}(w_1, \ldots, w_d).$$



- Security becomes a *combinatorial* property, decidable in principle.
- The abstraction is faithful: probing security implies noisy-leakage security (Duc et al. 2014).

The problem

Exact verification of d-probing security
Decide, for *every* one of the $\sum_{i \leq d} \binom{|\mathcal{W}|}{i}$ observation sets O, whether it is *independent of the secrets*:

$$\mathcal{L}_{\mathsf{a}}(O) = \mathcal{L}_{\mathsf{b}}(O) \qquad \text{for all } \mathsf{a}, \mathsf{b} \in \{0,1\}^{\mathcal{S}}.$$

- Secure $\Rightarrow$ every observation set accounted for,
- Insecure $\Rightarrow$ a specific leaking observation as witness.

The problem

Exact verification of d-probing security
Decide, for *every* one of the $\sum_{i \leq d} \binom{|\mathcal{W}|}{i}$ observation sets O, whether it is *independent of the secrets*:

$$\mathcal{L}_{\mathsf{a}}(O) \;=\; \mathcal{L}_{\mathsf{b}}(O) \qquad \text{for all } \mathsf{a}, \mathsf{b} \in \{0,1\}^{\mathcal{S}}.$$

- Secure $\Rightarrow$ every observation set accounted for,
- Insecure $\Rightarrow$ a specific leaking observation as witness.

- Two difficulties arise: a *single* probe already asks a question about a distribution over all of $\{0,1\}^{\mathcal{R}}$,
- and there are combinatorially many probes to settle.

How Hard Is One Probe?

Warm-up: three views of secret-independence

Fix a probe O (a tuple w of wires). Write $\mathcal{L}_{\mathsf{a}}(O)$ for its distribution over a uniform tape $\rho \in \{0,1\}^{\mathcal{R}}$, and $N_{\mathsf{a}}(o) = |\{\rho : O(\rho; \mathsf{a}) = o\}|$ for the tape counts.

1. **Distributional.** $\mathcal{L}_{\mathsf{a}}(O) = \mathcal{L}_{\mathsf{b}}(O)$ for all secrets a, b. *(the definition)*

Warm-up: three views of secret-independence

Fix a probe O (a tuple w of wires). Write $\mathcal{L}_{\mathsf{a}}(O)$ for its distribution over a uniform tape $\rho \in \{0,1\}^{\mathcal{R}}$, and $N_{\mathsf{a}}(o) = |\{\rho : O(\rho; \mathsf{a}) = o\}|$ for the tape counts.

1. **Distributional.** $\mathcal{L}_{\mathsf{a}}(O) = \mathcal{L}_{\mathsf{b}}(O)$ for all secrets a, b. *(the definition)*
2. **Counting.** $N_{\mathsf{a}}(o) = N_{\mathsf{b}}(o)$ for every observation o, i.e. two histograms of integers coincide.

   $\rightarrow$ *this section*

Warm-up: three views of secret-independence

Fix a probe O (a tuple w of wires). Write $\mathcal{L}_{\mathsf{a}}(O)$ for its distribution over a uniform tape $\rho \in \{0,1\}^{\mathcal{R}}$, and $N_{\mathsf{a}}(o) = |\{\rho : O(\rho; \mathsf{a}) = o\}|$ for the tape counts.

1. **Distributional.** $\mathcal{L}_{\mathsf{a}}(O) = \mathcal{L}_{\mathsf{b}}(O)$ for all secrets a, b. *(the definition)*
2. **Counting.** $N_{\mathsf{a}}(o) = N_{\mathsf{b}}(o)$ for every observation o, i.e. two histograms of integers coincide.
   $\rightarrow$ *this section*
3. **Coupling.** there exists a bijection Φ of the tapes such that $O(\rho; \mathsf{a}) = O(\Phi(\rho); \mathsf{b})$.
   $\rightarrow$ *next section*

Counting is intractable

Theorem (thesis, Ch. 4)
Computing one tape count $N_{\mathsf{a}}(o)$ is $\#\mathrm{P}$-complete (Valiant 1979). And deciding $\mathcal{L}_{\mathsf{a}}(O) = \mathcal{L}_{\mathsf{b}}(O)$ is $\mathrm{C_{=}P}$-complete (Wagner 1986).

Counting is intractable

Theorem (thesis, Ch. 4)
Computing one tape count $N_{\mathsf{a}}(o)$ is $\#\mathrm{P}$-complete (Valiant 1979). And deciding $\mathcal{L}_{\mathsf{a}}(O) = \mathcal{L}_{\mathsf{b}}(O)$ is $\mathrm{C}_{=}\mathrm{P}$-complete (Wagner 1986).

- Hardness: reduction from ExactSat. One secret bit and one probed wire already encodes it. (construction @ the appendix slides)
- So the hardness is intrinsic; no feature of masking causes it.

What the placement tells

- Deciding never costs more than counting ($\mathrm{C_=P} \subseteq \mathrm{P^{\#P}}$), and no polynomial exact decision exists unless $\mathrm{P} = \mathrm{NP}$.
- Both classes sit *above the polynomial hierarchy* (due to Toda). A single probe is already there, and full verification universally quantifies over exponentially many such instances.

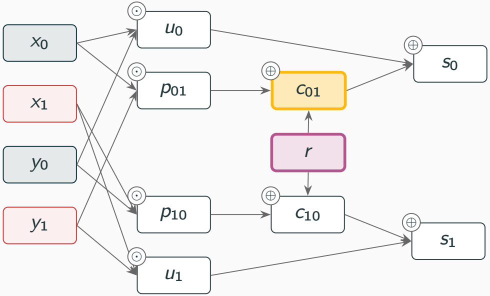
Consequence

We do not compute counts. We certify *constructively*: exhibit a relabelling of the randomness under which the secrets visibly cancel — a witness without counting, at the price of completeness!

Security Is a Coupling

One substitution, on the running example

Probe the masked cross term: $c_{01} = x_0 \odot y_1 \oplus r$.

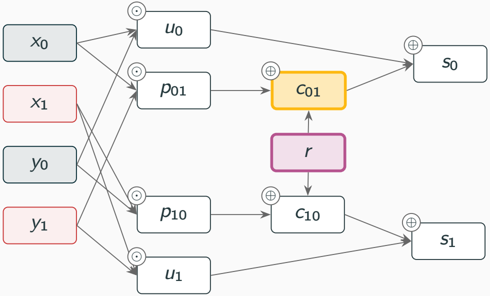


Apply the substitution $\sigma = \big(r \mapsto x_0 \odot y_1 \oplus r\big)$:

$$c_{01}\,\sigma \;=\; x_0 \odot y_1 \oplus \big(x_0 \odot y_1 \oplus r\big) \;=\; r.$$

One substitution, on the running example

Probe the masked cross term: $c_{01} = x_0 \odot y_1 \oplus r$.



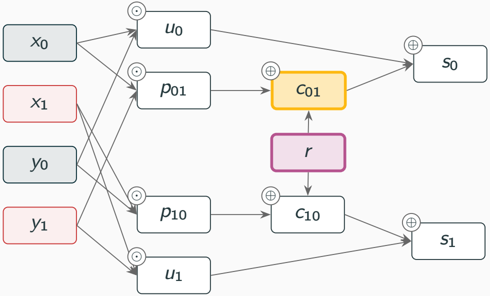
Apply the substitution $\sigma = (r \mapsto x_0 \odot y_1 \oplus r)$:

$$c_{01}\,\sigma \;=\; x_0 \odot y_1 \oplus (x_0 \odot y_1 \oplus r) \;=\; r.$$

- The probe is now *syntactically secret-free*: it is uniform, whatever the secrets are.

One substitution, on the running example

Probe the masked cross term: $c_{01} = x_0 \odot y_1 \oplus r$.



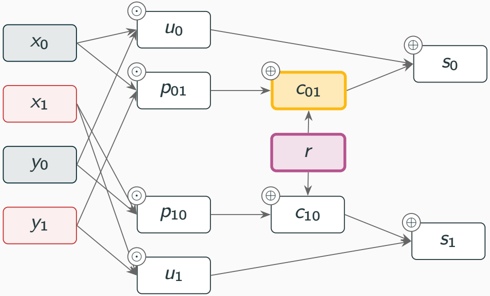
Apply the substitution $\sigma = (r \mapsto x_0 \odot y_1 \oplus r)$:

$$c_{01}\,\sigma \;=\; x_0 \odot y_1 \oplus (x_0 \odot y_1 \oplus r) \;=\; r.$$

- The probe is now *syntactically secret-free*: it is uniform, whatever the secrets are.
- And σ induces a *bijection* of the tape space, translating the coordinate r, a *shear*.

One substitution, on the running example

Probe the masked cross term: $c_{01} = x_0 \odot y_1 \oplus r$.



Apply the substitution $\sigma = (r \mapsto x_0 \odot y_1 \oplus r)$:

$$c_{01}\,\sigma \;=\; x_0 \odot y_1 \oplus (x_0 \odot y_1 \oplus r) \;=\; r.$$

- The probe is now *syntactically secret-free*: it is uniform, whatever the secrets are.
- And σ induces a *bijection* of the tape space, translating the coordinate r, a *shear*.
- The context $x_0 \odot y_1$ mentions a share, so the *one* syntactic step induces a *family* of shears — fixing the secrets a picks $\widehat{\sigma}^{\mathrm{a}}$.

Key identity: $(\mathrm{w}\sigma)(\rho; \mathrm{a}) = \mathrm{w}(\widehat{\sigma}^{\mathrm{a}}(\rho); \mathrm{a})$, syntax and semantics agree, for every a.

Couplings

To prove two distributions equal *without computing them*, link their sources ((Barthe et al. 2017) for probabilistic programs). Our two laws arise from the *same* uniform tape, so we link the tapes.

Definition (coupling function)
A *coupling function* from a to b for a probe O is a bijection $\Phi : \{0,1\}^{\mathcal{R}} \to \{0,1\}^{\mathcal{R}}$ with

$$O(\rho; \mathsf{a}) \;=\; O\big(\Phi(\rho); \mathsf{b}\big) \qquad \text{for every tape } \rho.$$

- Bijectivity preserves the uniform law; the identity says a run under a is reproduced, observation for observation, under b. Hence we have equal laws.

Security *is* a coupling

Theorem
A probe O is secret-independent *iff* for every ordered pair a, b there is a coupling function from a to b for O.

Proof sketch.

- ($\Leftarrow$) A bijection merely re-indexes the tape space, so the counts behind the two laws agree.
- ($\Rightarrow$) Equal laws mean the outcome-fibres $\{\rho : O(\rho; \mathsf{a}) = o\}$ come in matching sizes; match them block by block and glue. $\square$

(full sketch: backup slides)

Security *is* a coupling

Theorem
A probe O is secret-independent *iff* for every ordered pair a, b there is a coupling function from a to b for O.

Proof sketch.

- ($\Leftarrow$) A bijection merely re-indexes the tape space, so the counts behind the two laws agree.
- ($\Rightarrow$) Equal laws mean the outcome-fibres $\{\rho : O(\rho; \mathsf{a}) = o\}$ come in matching sizes; match them block by block and glue. $\square$

(full sketch: backup slides)

This is an existence statement and not a procedure
Matching the fibres needs the counts $N_{\mathsf{a}}(o)$, which are $\#\mathrm{P}$-hard. The theorem guarantees a coupling; it does not find one.

From substitutions to couplings

A derivation $\sigma_1, \ldots, \sigma_K$ ending secret-free is a *recipe*: run under secrets a, it becomes the tape bijection $\varphi_{\mathsf{a}} = \widehat{\sigma_1}^{\mathsf{a}} \circ \cdots \circ \widehat{\sigma_K}^{\mathsf{a}}$. Applied syntactically, it yields $\overline{O} = O\sigma_1 \ldots \sigma_K$.

$$\begin{tikzcd}
O(\,\cdot\,; \mathsf{a}) \arrow[dr, "\varphi_{\mathsf{a}}"'] & & O(\,\cdot\,; \mathsf{b}) \arrow[dl, "\varphi_{\mathsf{b}}"] \\
& \overline{O} &
\end{tikzcd}$$

(secret-free normal form)

From substitutions to couplings

A derivation $\sigma_1, \ldots, \sigma_K$ ending secret-free is a *recipe*: run under secrets a, it becomes the tape bijection $\varphi_{\mathsf{a}} = \widehat{\sigma_1}^{\mathsf{a}} \circ \cdots \circ \widehat{\sigma_K}^{\mathsf{a}}$. Applied syntactically, it yields $\overline{O} = O\sigma_1 \ldots \sigma_K$.

$$\begin{tikzcd}
O(\cdot\,;\mathsf{a}) \arrow[rr, dashed, "\varphi_{\mathsf{b}} \circ \varphi_{\mathsf{a}}^{-1} = \Phi"] \arrow[dr, "\varphi_{\mathsf{a}}"'] & & O(\cdot\,;\mathsf{b}) \arrow[dl, "\varphi_{\mathsf{b}}"] \\
& \overline{O} \quad \text{(secret-free normal form)} &
\end{tikzcd}$$

- Every run is a re-labelled view of the *same* secret-free experiment, so any two runs link through it: a coupling for *every* pair of secrets, from one derivation.

From substitutions to couplings

A derivation $\sigma_1, \ldots, \sigma_K$ ending secret-free is a *recipe*: run under secrets a, it becomes the tape bijection $\varphi_{\mathsf{a}} = \widehat{\sigma_1}^{\mathsf{a}} \circ \cdots \circ \widehat{\sigma_K}^{\mathsf{a}}$. Applied syntactically, it yields $\overline{O} = O\sigma_1 \ldots \sigma_K$.

$$\begin{tikzcd}
O(\cdot\,;\mathsf{a}) \arrow[rr, dashed, "\varphi_{\mathsf{b}} \circ \varphi_{\mathsf{a}}^{-1} = \Phi"] \arrow[dr, "\varphi_{\mathsf{a}}"'] & & O(\cdot\,;\mathsf{b}) \arrow[dl, "\varphi_{\mathsf{b}}"] \\
& \overline{O} \ \text{(secret-free normal form)} &
\end{tikzcd}$$

- Every run is a re-labelled view of the *same* secret-free experiment, so any two runs link through it: a coupling for *every* pair of secrets, from one derivation.
- On DOM-AND the recipe is the single substitution $r \mapsto x_0 \odot y_1 \oplus r$; its context $x_0 \odot y_1$ is what makes φ depend on the secrets.

From substitutions to couplings

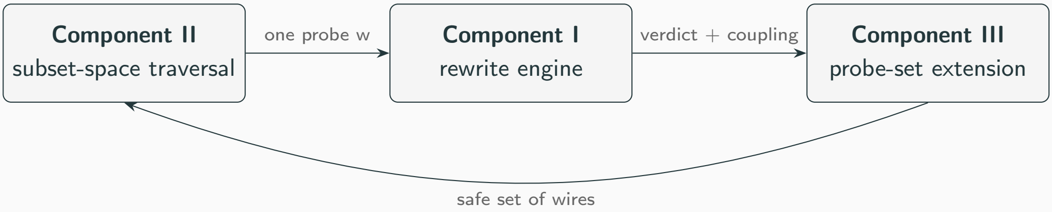
A derivation $\sigma_1, \ldots, \sigma_K$ ending secret-free is a *recipe*: run under secrets a, it becomes the tape bijection $\varphi_{\mathsf{a}} = \widehat{\sigma_1}^{\mathsf{a}} \circ \cdots \circ \widehat{\sigma_K}^{\mathsf{a}}$. Applied syntactically, it yields $\overline{O} = O\sigma_1 \ldots \sigma_K$.

$$\begin{tikzcd}
O(\cdot\,;\mathsf{a}) \arrow[rr, dashed, "\varphi_{\mathsf{b}} \circ \varphi_{\mathsf{a}}^{-1} = \Phi"] \arrow[dr, "\varphi_{\mathsf{a}}"'] & & O(\cdot\,;\mathsf{b}) \arrow[dl, "\varphi_{\mathsf{b}}"] \\
& \overline{O} \quad \text{(secret-free normal form)} &
\end{tikzcd}$$

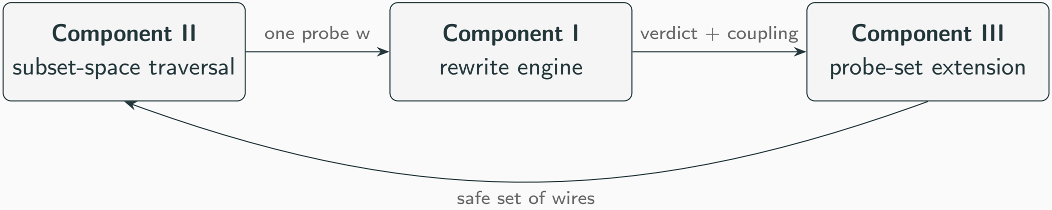
- Every run is a re-labelled view of the *same* secret-free experiment, so any two runs link through it: a coupling for *every* pair of secrets, from one derivation.
- On DOM-AND the recipe is the single substitution $r \mapsto x_0 \odot y_1 \oplus r$; its context $x_0 \odot y_1$ is what makes φ depend on the secrets.
- So a derivation is a **reusable certificate**: the design principle of Coppula.

Coppula: Three Components

Pipeline overview

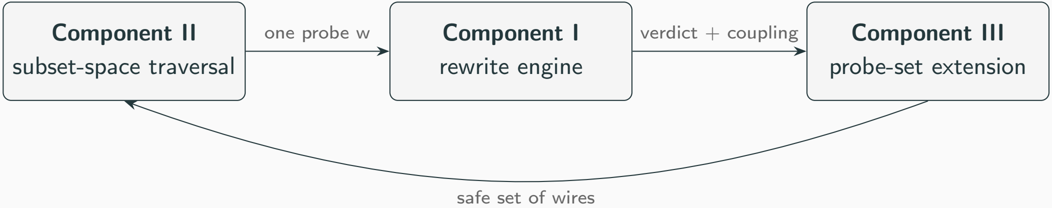


Pipeline overview



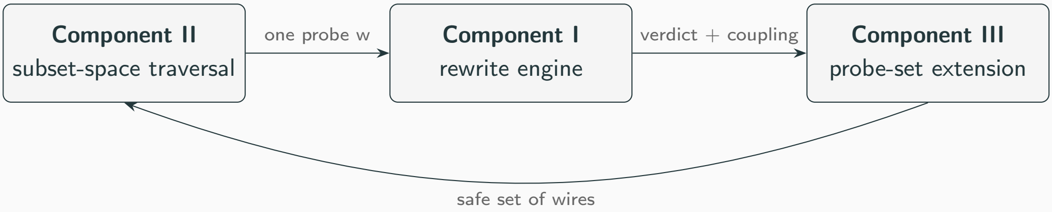
- **I** decides a *single* probe, by construction.

Pipeline overview



- **I** decides a *single* probe, by construction.
- **II** sweeps *every* d-subset without explicitly enumerating them.

Pipeline overview



- **I** decides a *single* probe, by construction.
- **II** sweeps *every* d-subset without explicitly enumerating them.
- **III** amplifies one certificate into a whole *safe set*, shrinking the space **II** must sweep.

Component I: the rewrite engine

Decides one probe w by substituting randoms away, one at a time, driving the observation toward a secret-free form.

- **Tiered.** A cheap single-occurrence rule first (r occurs once across w: substitute, done); with a complete substitution loop behind it.

Component I: the rewrite engine

Decides one probe w by substituting randoms away, one at a time, driving the observation toward a secret-free form.

- **Tiered.** A cheap single-occurrence rule first (r occurs once across w: substitute, done); with a complete substitution loop behind it.
- **Sound.** Every step is a shear, so any Secure verdict carries a genuine coupling (Theorem, Ch. 5).

Component I: the rewrite engine

Decides one probe w by substituting randoms away, one at a time, driving the observation toward a secret-free form.

- **Tiered.** A cheap single-occurrence rule first (r occurs once across w: substitute, done); with a complete substitution loop behind it.
- **Sound.** Every step is a shear, so any Secure verdict carries a genuine coupling (Theorem, Ch. 5).
- **Incomplete, knowingly.** A probe can be independent for reasons no relabelling of this kind captures. This is the price of the hardness result.

Component I: the rewrite engine

Decides one probe w by substituting randoms away, one at a time, driving the observation toward a secret-free form.

- **Tiered.** A cheap single-occurrence rule first (r occurs once across w: substitute, done); with a complete substitution loop behind it.
- **Sound.** Every step is a shear, so any Secure verdict carries a genuine coupling (Theorem, Ch. 5).
- **Incomplete, knowingly.** A probe can be independent for reasons no relabelling of this kind captures. This is the price of the hardness result.

On the example: $c_{01} \xrightarrow{r \mapsto x_{\mathbf{0}} \odot y_{\mathbf{1}} \oplus r} r$, one step, Secure, coupling recorded.

Some masks are buried inside *products*, where substitution cannot reach them.

- Substituting $r \mapsto e \oplus r$ needs r to occur as a *summand*; in $u \odot r \oplus u \odot v$ every occurrence of r is trapped inside a product.

Component I: why factoring

Some masks are buried inside *products*, where substitution cannot reach them.

- Substituting $r \mapsto e \oplus r$ needs r to occur as a *summand*; in $u \odot r \oplus u \odot v$ every occurrence of r is trapped inside a product.
- **Multiplicative factoring** pulls the common factor out,

$$u \odot r \oplus u \odot v \longmapsto u \odot (r \oplus v),$$

and the hidden random surfaces as a summand again, ready to substitute.

Component I: why factoring

Some masks are buried inside *products*, where substitution cannot reach them.

- Substituting $r \mapsto e \oplus r$ needs r to occur as a *summand*; in $u \odot r \oplus u \odot v$ every occurrence of r is trapped inside a product.
- **Multiplicative factoring** pulls the common factor out,

$$u \odot r \;\oplus\; u \odot v \;\longmapsto\; u \odot (r \oplus v),$$

  and the hidden random surfaces as a summand again, ready to substitute.
- Applied lazily between substitution attempts, until the probe discharges or no rule applies.

Component II: the representation

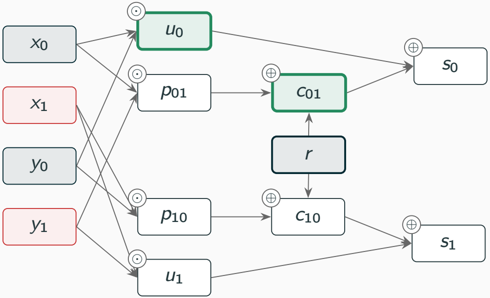
Certifying each d-subset in turn is infeasible. So the traversal never lists probes; it holds compact descriptions of whole families, in the manner of maskVerif (Barthe et al. 2019).

Factors and worklists
A *factor* (c, W) stands for "choose c wires from the set W". A *worklist* $\langle(c_1, W_1), \ldots, (c_m, W_m)\rangle$, the W_j disjoint, denotes all probes drawing c_j wires from each W_j: it holds $\prod_j \binom{|W_j|}{c_j}$ probes in a few words.

On DOM-AND *(order* 2*, for exposition)*: the probe targets are the eight computed wires $\mathcal{W} = \{u_0, u_1, p_{01}, p_{10}, c_{01}, c_{10}, s_0, s_1\}$, and the whole verification task is the single factor

$$\big\langle(2, \mathcal{W})\big\rangle, \qquad \text{denoting all } \binom{8}{2} = 28 \text{ pairs.}$$

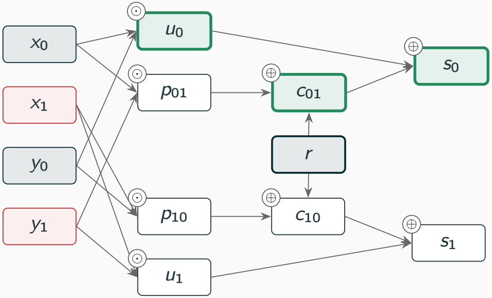
Component II: one recursion step, on our example



the illustrated probe set, in green

1. **Prune?** Discharge the union of all 8 wires at once. It fails here (the gadget leaks at order 2), so we descend.

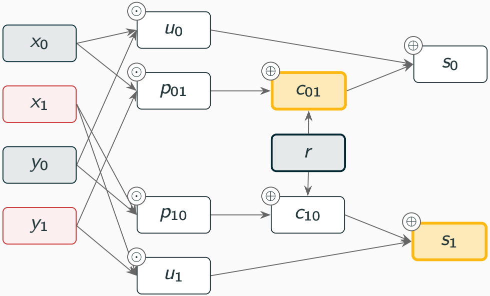
Component II: one recursion step, on our example



the illustrated safe set, in green

1. **Prune?** Discharge the union of all 8 wires at once. It fails here (the gadget leaks at order 2), so we descend.
2. **Certify a representative probe**, say $\{u_0, c_{01}\}$, and grow it (Component III) into a safe set, here $\{u_0, c_{01}, s_0\}$.

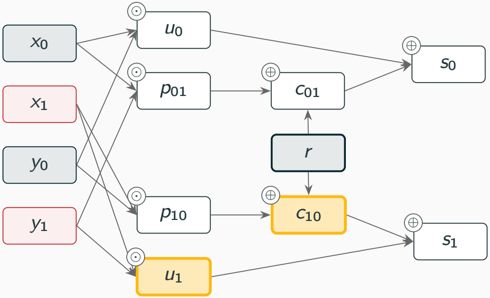
Component II: one recursion step, on our example



a cross-term factor, in yellow

1. **Prune?** Discharge the union of all 8 wires at once. It fails here (the gadget leaks at order 2), so we descend.
2. **Certify a representative probe**, say $\{u_0, c_{01}\}$, and grow it (Component III) into a safe set, here $\{u_0, c_{01}, s_0\}$.
3. **Split** along safe / unsafe: $28 = \binom{3}{2} + 3 \cdot 5 + \binom{5}{2} = 3 + 15 + 10$ (all-safe, mixed, all-unsafe). *Skip* the all-safe cell and recurse on the 25 that remain.

Component II: one recursion step, on our example



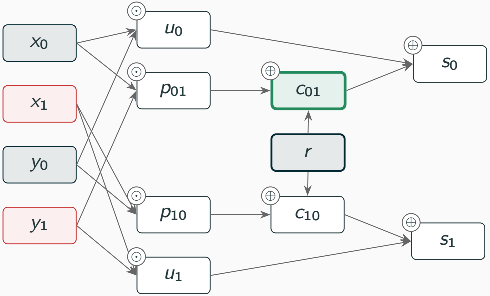
a unsafe factor, in yellow

1. **Prune?** Discharge the union of all 8 wires at once. It fails here (the gadget leaks at order 2), so we descend.
2. **Certify a representative probe**, say $\{u_0, c_{01}\}$, and grow it (Component III) into a safe set, here $\{u_0, c_{01}, s_0\}$.
3. **Split** along safe / unsafe: $28 = \binom{3}{2} + 3 \cdot 5 + \binom{5}{2} = 3 + 15 + 10$ (all-safe, mixed, all-unsafe). *Skip* the all-safe cell and recurse on the 25 that remain.

Theorem (exhaustiveness, Ch. 5). Read set-wise, Vandermonde's identity says the cells *partition* the probes: every d-subset is visited exactly once.

Component III: growing safe sets

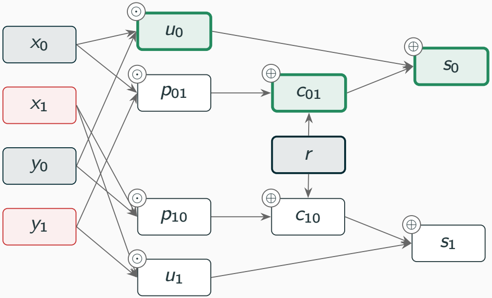
A discharged probe is a coupling, but can the *same* coupling cover more wires?



- A wire u joins the safe set of coupling φ when the composite $u \circ \varphi$ is secret-free.

Component III: growing safe sets

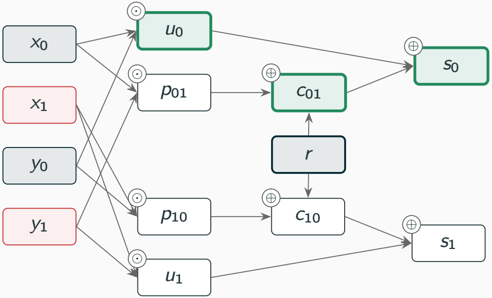
A discharged probe is a coupling, but can the *same* coupling cover more wires?



- A wire u joins the safe set of coupling φ when the composite $u \circ \varphi$ is secret-free.
- Here $\varphi = (r \mapsto r \oplus x_0 \odot y_1)$ also blinds u_0 and s_0, as $u_0 \circ \varphi = x_0 \odot y_0$ and $s_0 \circ \varphi = x_0 \odot y_0 \oplus r$.

Component III: growing safe sets

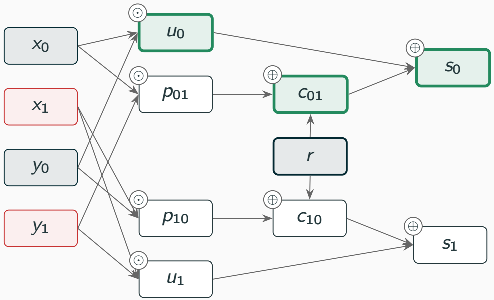
A discharged probe is a coupling, but can the *same* coupling cover more wires?



- A wire u joins the safe set of coupling φ when the composite $u \circ \varphi$ is secret-free.
- Here $\varphi = (r \mapsto r \oplus x_0 \odot y_1)$ also blinds u_0 and s_0, as $u_0 \circ \varphi = x_0 \odot y_0$ and $s_0 \circ \varphi = x_0 \odot y_0 \oplus r$.
- Exact test via *ANF*: a function depends on a variable iff it occurs in its normal form.

Component III: growing safe sets

A discharged probe is a coupling, but can the *same* coupling cover more wires?



- A wire u joins the safe set of coupling φ when the composite $u \circ \varphi$ is secret-free.
- Here $\varphi = (r \mapsto r \oplus x_0 \odot y_1)$ also blinds u_0 and s_0, as $u_0 \circ \varphi = x_0 \odot y_0$ and $s_0 \circ \varphi = x_0 \odot y_0 \oplus r$.
- Exact test via *ANF*: a function depends on a variable iff it occurs in its normal form.
- One discharge settles the whole safe set; Component II needs less traversal.

Component III: three mechanisms

- **Closure** — build-time, entropy-style deduction ($H(u \mid \mathrm{w}) = 0$). Local deduction on a scattered pool: cheap but weak.

Proposition (nesting, Ch. 5)
closure $\subseteq$ replay $\subseteq$ coupling — every wire the cheaper mechanisms admit, the coupling admits too. Coppula adopts the coupling approach.

Component III: three mechanisms

- **Closure** — build-time, entropy-style deduction ($H(u \mid \mathrm{w}) = 0$). Local deduction on a scattered pool: cheap but weak.
- **Replay** — re-run the derivation on the candidate wire (Barthe et al. 2015). An imperfect fit for us because we *factor*, and factoring is not a replayable step.

Proposition (nesting, Ch. 5)
closure $\subseteq$ replay $\subseteq$ coupling — every wire the cheaper mechanisms admit, the coupling admits too. Coppula adopts the coupling approach.

Component III: three mechanisms

- **Closure** — build-time, entropy-style deduction ($H(u \mid \mathrm{w}) = 0$). Local deduction on a scattered pool: cheap but weak.
- **Replay** — re-run the derivation on the candidate wire (Barthe et al. 2015). An imperfect fit for us because we *factor*, and factoring is not a replayable step.
- **Coupling** — apply the recorded tape bijection and test the ANF endpoint. Exact, and independent of how the derivation was found.

Proposition (nesting, Ch. 5)
closure $\subseteq$ replay $\subseteq$ coupling — every wire the cheaper mechanisms admit, the coupling admits too. Coppula adopts the coupling approach.

Given the coupling φ and a candidate wire u from the pool: is $u \circ \varphi$ secret-free?

Phase 1 — fast reject (probabilistic, may only *reject*)

Sample tapes; evaluate $u \circ \varphi$ twice per tape, secrets drawn at random vs. all zero. A disagreement proves $u \circ \varphi$ moves with the secrets: drop u. Agreement proves nothing.

Component III: the test

Given the coupling φ and a candidate wire u from the pool: is $u \circ \varphi$ secret-free?

Phase 1 — fast reject (probabilistic, may only *reject*)
Sample tapes; evaluate $u \circ \varphi$ twice per tape, secrets drawn at random vs. all zero. A disagreement proves $u \circ \varphi$ moves with the secrets: drop u. Agreement proves nothing.

Phases 2–3 — exact ($\mathbb{F}_2$ cancellation, then ANF)
A function depends on a variable iff the variable occurs in its unique multilinear normal form. So admit u exactly when no secret survives in the ANF of $u \circ \varphi$ (Lemma, Ch. 5).

Component III: the test

Given the coupling φ and a candidate wire u from the pool: is $u \circ \varphi$ secret-free?

Phase 1 — fast reject (probabilistic, may only *reject*)
Sample tapes; evaluate $u \circ \varphi$ twice per tape, secrets drawn at random vs. all zero. A disagreement proves $u \circ \varphi$ moves with the secrets: drop u. Agreement proves nothing.

Phases 2–3 — exact ($\mathbb{F}_2$ cancellation, then ANF)
A function depends on a variable iff the variable occurs in its unique multilinear normal form. So admit u exactly when no secret survives in the ANF of $u \circ \varphi$ (Lemma, Ch. 5).

Why blinded wires are safe to add. For admitted u, $u(\varphi_{\mathsf{a}}(\rho); \mathsf{a}) = \overline{u}(\rho)$ depends on ρ alone; as φ_{a} preserves the uniform law, the *joint* law of the safe set is the same under every secret (Theorem, Ch. 5).

Implementation & Evaluation

Implementation details

Coppula is implemented in **Lean 4**.

- **Hash-consed DAG** expression representation with canonicalization, where structural sharing makes equality checks and substitutions cheap.
- **Multiplicative factoring** as a normal-form pass over the DAG, applied lazily.
- **Bit-sliced ANF endpoint test** for the extension: word-parallel evaluation decides secret-freeness of $u \circ \varphi$ exactly.

Implementation details

Coppula is implemented in **Lean 4**.

- **Hash-consed DAG** expression representation with canonicalization, where structural sharing makes equality checks and substitutions cheap.
- **Multiplicative factoring** as a normal-form pass over the DAG, applied lazily.
- **Bit-sliced ANF endpoint test** for the extension: word-parallel evaluation decides secret-freeness of $u \circ \varphi$ exactly.

Design and proofs are implementation-agnostic; these choices simply make algorithms fast.

Evaluation: outcomes

Forty configurations at orders 1–4: ISW multiplications, DOM and TI ANDs, refresh gadgets, Keccak χ (DOM and TI sharings), and the TI S-boxes and quadratic sharings of Bilgin et al. Every verdict matches the ground truth, and every Insecure comes with a concrete witness observation.

- **DOM-AND** (2 shares): Secure at order 1, the witness pair $\{c_{01}, c_{10}\}$ at order 2 (our running example).
- **Four-share** $x \oplus yz$ **TI** (Bilgin et al. 2015): certified secure at orders 1 and 2, insecure at order 3. The *expected* behaviour of a first-order threshold implementation.

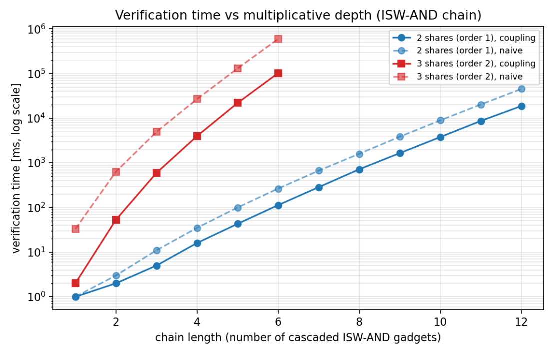
Evaluation: outcomes

Forty configurations at orders 1–4: ISW multiplications, DOM and TI ANDs, refresh gadgets, Keccak χ (DOM and TI sharings), and the TI S-boxes and quadratic sharings of Bilgin et al. Every verdict matches the ground truth, and every Insecure comes with a concrete witness observation.

- **DOM-AND** (2 shares): Secure at order 1, the witness pair $\{c_{01}, c_{10}\}$ at order 2 (our running example).
- **Four-share** $x \oplus yz$ **TI** (Bilgin et al. 2015): certified secure at orders 1 and 2, insecure at order 3. The *expected* behaviour of a first-order threshold implementation.
- **The factoring family** ($d+1$ quadratic S-boxes, Q^3/Q^4): randomness-free sharings whose certification must factor first. A factoring-free, maskVerif-style engine false-reports Insecure on five; Coppula verifies every one.

(Full extension-comparison table @ the appendix slides.)

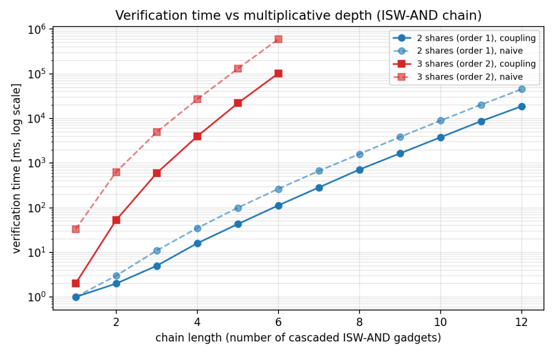
Evaluation: is the extension worth it?



Verification time versus circuit depth.

- Coupling matches replay's yield *to the digit* wherever replay is right (five-share ISW: 4735 discharges, 8078 free wires on both), and keeps the correct verdict on the five rows where replay false-reports Insecure.

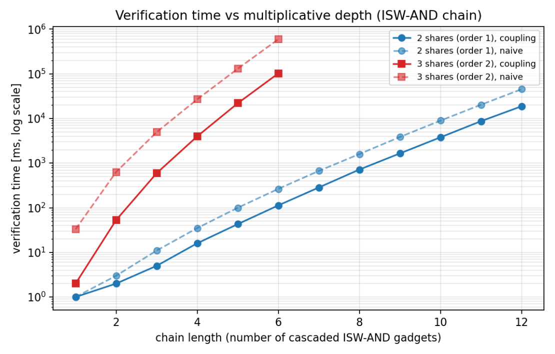
Evaluation: is the extension worth it?



Verification time versus circuit depth.

- Coupling matches replay's yield *to the digit* wherever replay is right (five-share ISW: 4735 discharges, 8078 free wires on both), and keeps the correct verdict on the five rows where replay false-reports Insecure.
- Closures keep the verdicts but lose the yield: 14 vs. 537 free wires on Fides, 632,834 vs. 4735 discharges on five-share ISW, because the split degrades toward enumeration.

Evaluation: is the extension worth it?



Verification time versus circuit depth.

- Coupling matches replay's yield *to the digit* wherever replay is right (five-share ISW: 4735 discharges, 8078 free wires on both), and keeps the correct verdict on the five rows where replay false-reports Insecure.
- Closures keep the verdicts but lose the yield: 14 vs. 537 free wires on Fides, 632,834 vs. 4735 discharges on five-share ISW, because the split degrades toward enumeration.
- The extension pays for itself: Fides certifies 537 wires from 55 discharges; five-share ISW falls from $\sim$609 s (no extension) to under 2 s.

Wrap-up

Limitations and future work

- **Model.** Our model is value-based leakage only: glitches and transitions are out of scope; so is composition. So we show single-circuit, exact probing security.

Limitations and future work

- **Model.** Our model is value-based leakage only: glitches and transitions are out of scope; so is composition. So we show single-circuit, exact probing security.
- **Larger fields.** The core (shears, couplings, ANF) is already field-agnostic; lifting the implementation beyond $\mathbb{F}_2$ is engineering plus a normal-form choice. (Also, the power of Schwartz–Zippel can come into picture here.)

Limitations and future work

- **Model.** Our model is value-based leakage only: glitches and transitions are out of scope; so is composition. So we show single-circuit, exact probing security.
- **Larger fields.** The core (shears, couplings, ANF) is already field-agnostic; lifting the implementation beyond $\mathbb{F}_2$ is engineering plus a normal-form choice. (Also, the power of Schwartz–Zippel can come into picture here.)
- **Composing couplings.** for unions of safe sets using couplings as a guide.

Limitations and future work

- **Model.** Our model is value-based leakage only: glitches and transitions are out of scope; so is composition. So we show single-circuit, exact probing security.
- **Larger fields.** The core (shears, couplings, ANF) is already field-agnostic; lifting the implementation beyond $\mathbb{F}_2$ is engineering plus a normal-form choice. (Also, the power of Schwartz–Zippel can come into picture here.)
- **Composing couplings.** for unions of safe sets using couplings as a guide.
- **More.** smarter factoring heuristics; exploiting circuit symmetries to quotient the subset space.

Contributions

1. **Placement.** We show that: counting is $\#\mathrm{P}$-complete, deciding is $\mathrm{C_{=}P}$-complete, i.e. they sit above the polynomial hierarchy, ruling out the counting route.

Contributions

1. **Placement.** We show that: counting is $\#\mathrm{P}$-complete, deciding is $\mathrm{C}_{=}\mathrm{P}$-complete, i.e. they sit above the polynomial hierarchy, ruling out the counting route.
2. **A rigorous coupling account of probing security.** Security *is* the existence of a family of coupling functions, the family is a groupoid, and substitutions construct it.

Contributions

1. **Placement.** We show that: counting is $\#\mathrm{P}$-complete, deciding is $\mathrm{C}_{=}\mathrm{P}$-complete, i.e. they sit above the polynomial hierarchy, ruling out the counting route.
2. **A rigorous coupling account of probing security.** Security *is* the existence of a family of coupling functions, the family is a groupoid, and substitutions construct it.
3. **Coppula.** Made up of a sound rewrite engine (with factoring), an exhaustive subset-space traversal, and a probe-set extension with a proven mechanism hierarchy closure $\subseteq$ replay $\subseteq$ coupling.

Contributions

1. **Placement.** We show that: counting is $\#\mathrm{P}$-complete, deciding is $\mathrm{C}_{=}\mathrm{P}$-complete, i.e. they sit above the polynomial hierarchy, ruling out the counting route.
2. **A rigorous coupling account of probing security.** Security *is* the existence of a family of coupling functions, the family is a groupoid, and substitutions construct it.
3. **Coppula.** Made up of a sound rewrite engine (with factoring), an exhaustive subset-space traversal, and a probe-set extension with a proven mechanism hierarchy closure $\subseteq$ replay $\subseteq$ coupling.
4. **Evaluation.** We present exact verdicts, with witnesses, across a suite of masked gadgets, and the coupling extension wins in coverage.

Thank you!

Questions?

Backup

Backup: extension comparison (I/IV)

Each entry: Coupling / Replay / Closure. Verdict S = Secure, I = Insecure; † marks replay's false-Insecures.

Circuit	d	Verdict	Time [ms]	Discharges	Free wires
ISW multiplication					
2 sh.	1	S/S/S	0/0/1	7/7/15	2/2/0
3 sh.	2	S/S/S	3/1/34	34/34/309	31/31/8
4 sh.	3	S/S/S	46/37/3190	338/338/10770	437/437/439
5 sh.	4	S/S/S	1125/845/409472	4735/4735/632834	8078/8078/37513
DOM AND					
2 sh.	1	S/S/S	1/0/0	11/11/15	2/2/0
2 sh.	2	I/I/I	0/0/1	2/2/6	0/0/2
3 sh.	2	S/S/S	5/4/35	88/88/325	44/44/7
4 sh.	3	S/S/S	104/90/3419	859/859/13291	784/784/532

Backup: extension comparison (II/IV)

Circuit	d	Verdict	Time [ms]	Discharges	Free wires
Trichina AND					
well-formed	1	S/S/S	0/1/1	11/11/15	2/2/0
mis-associated	1	I/I/I	1/0/1	10/10/12	1/1/0
TI AND					
3 sh.	1	S/S/S	0/0/1	9/9/27	9/9/0
3 sh.	2	I/I/I	1/1/1	10/10/10	5/5/2
Mask refreshing					
additive, 3 sh.	2	S/S/S	0/0/0	6/6/8	1/1/0
additive, 4 sh.	3	S/S/S	1/0/1	11/11/17	6/6/0
ISW, 3 sh.	2	S/S/S	0/0/0	6/6/10	4/4/0
ISW, 4 sh.	3	S/S/S	0/1/2	12/12/51	24/24/0
Keccak χ					
DOM, 2 sh.	1	S/S/S	14/11/29	49/49/107	29/29/5
DOM, 3 sh.	2	S/S/S	2807/1797/11563	2865/2865/14047	2671/2671/645
TI, 3 sh.	1	S/S/S	21/14/103	43/43/183	68/68/5
TI, 3 sh.	2	I/I/I	21/14/72	57/57/352	92/92/27

Backup: extension comparison (III/IV)

Circuit	d	Verdict	Time [ms]	Discharges	Free wires
TI S-boxes and quadratic sharings					
Q^4_{12} direct, 2 sh.	1	S/I†/S	2/2/5	27/26/47	16/16/10
Q^4_{12} TI, 3 sh.	1	S/S/S	5/4/25	25/25/83	30/30/6
Fides AB1, 4 sh.	1	S/S/S	1941/972/36461	55/55/1127	537/537/14
$x + yz$ direct, 3 sh.	1	S/S/S	1/0/3	7/7/29	10/10/0
$x + yz$ w/ CT, 3 sh.	1	S/S/S	1/1/3	11/11/29	9/9/0
$x + yz$, 4 sh.	2	S/S/S	11/8/170	78/78/767	115/115/20
$x + yz$, 4 sh.	3	I/I/I	11/7/8	102/102/72	99/99/15
$x + yz + xyz$, 4 sh.	1	S/S/S	45/18/175	31/31/167	63/63/1
xy virtual var.	1	S/S/S	3/1/17	9/9/55	23/23/0
xy virtual share	1	S/S/S	1/0/7	9/9/37	15/15/0
xy, 4$\rightarrow$3 sh.	1	S/S/S	1/1/3	11/11/25	8/8/0

Backup: extension comparison (IV/IV)

Circuit	d	Verdict	Time [ms]	Discharges	Free wires
$d{+}1$ **quadratic S-box family**					
Q^3_1	1	S/S/S	0/1/0	13/13/23	11/11/10
Q^3_2	1	S/I†/S	2/1/4	25/24/43	13/13/6
Q^3_3	1	I/I/I	4/2/7	18/18/32	22/22/2
Q^4_4	1	S/S/S	1/0/1	15/15/27	14/14/14
Q^4_{12}	1	S/I†/S	2/2/5	25/24/47	15/15/8
Q^4_{293}	1	S/I†/S	4/2/6	31/24/59	20/20/8
Q^4_{294}	1	S/S/S	1/1/3	23/23/39	16/16/12
Q^4_{299}	1	S/I†/S	5/2/13	35/18/75	26/22/8
Q^4_{300}	1	I/I/I	4/3/8	18/18/32	24/24/2

The two genuinely insecure classes, Q^3_3 and Q^4_{300}, are exactly the quadratic classes that admit no uniform three-share TI (Bilgin et al. 2015).

Backup: $\mathrm{C_{=}P}$ membership, sketch

Encode disagreement of the two laws as a single gap function:

$$f(C, O, \mathsf{a}, \mathsf{b}) \;=\; \sum_{o} \big(N_{\mathsf{a}}(o) - N_{\mathsf{b}}(o)\big)^2.$$

Each N is a $\#\mathrm{P}$ function; GapP is closed under subtraction and multiplication (Fenner et al. 1994); the sum has $2^k \le 2^d$ terms, constant for fixed d. A sum of squares vanishes iff every term does, so

$$\mathcal{L}_{\mathsf{a}}(O) = \mathcal{L}_{\mathsf{b}}(O) \iff f = 0,$$

and the problem is the vanishing set of a gap function, which is exactly $\mathrm{C_{=}P}$.

Backup: the ExactSat reduction

Reduce from the equality form of ExactSat: given formulas φ_0, φ_1 over variables $x_1, \ldots, x_m$, decide $\#\varphi_0 = \#\varphi_1$ ($\mathrm{C_{=}P}$-complete, (Wagner 1986)).

Build a circuit with one secret bit s, the x_i as randoms, and the single wire

$$w \;=\; \varphi_0 \;\oplus\; \big(s \odot (\varphi_0 \oplus \varphi_1)\big),$$

realising the formulas over $\oplus$ and $\odot$. Then w reads φ_0 when $s = 0$ and φ_1 when $s = 1$, so the one-bit probe $\{w\}$ is secret-independent *iff* $\#\varphi_0 = \#\varphi_1$.

The construction is polynomial, and one secret bit with one probed wire suffices: no feature of masking is what makes the problem hard.

Backup: security is a coupling, full sketch

($\Leftarrow$) Reindex by $\rho' = \varphi(\rho)$:

$$N_{\mathsf{a}}(o) = \big|\{\rho : O(\rho; \mathsf{a}) = o\}\big| = \big|\{\rho : O(\varphi(\rho); \mathsf{b}) = o\}\big| = N_{\mathsf{b}}(o).$$

($\Rightarrow$) Sort the tapes by outcome: $F_{\mathsf{a}}(o) = \{\rho : O(\rho; \mathsf{a}) = o\}$. As o varies these *fibres* partition $\{0,1\}^{\mathcal{R}}$, and equal laws mean matching fibres have equal size, $|F_{\mathsf{a}}(o)| = |F_{\mathsf{b}}(o)|$. So pick, for each o, any bijection $F_{\mathsf{a}}(o) \to F_{\mathsf{b}}(o)$; their union is a bijection φ of the whole tape space, and it sends every ρ to a tape with the *same* observation, giving us a coupling function. $\square$

Backup: replay vs. factoring

The derivations of Barthe et al. (2015) replay two rule kinds: Opt (substitution) and Conv (ANF normalisation) — for them, ANF is a *replayed rule*.

Our derivations interleave substitutions with *multiplicative factoring*, and a factoring step has no tape-level counterpart to replay. Replaying the substitution steps alone on a new wire can spuriously fail, producing false-Insecure verdicts (observed in the evaluation, the † rows).

Hence the coupling mechanism: apply the recorded bijection $\widehat{\sigma}_1 \circ \cdots \circ \widehat{\sigma}_k$ semantically and test the ANF endpoint — exact, and indifferent to how the derivation was found.

Backup: positioning

- **maskVerif** (Barthe et al. 2019): language-based, the traversal's main inspiration; replay-style extension; no factoring.
- **REBECCA / Fourier-analytic** (Bloem et al. 2018): correlation sets over Fourier coefficients; approximate in general.
- **SILVER** (Knichel et al. 2020): exact, BDD-based model counting; exhaustive per-probe distributions, different scaling profile.
- **Coppula**: exact like SILVER, constructive like maskVerif — and the certificate (the coupling) is itself an object with structure we exploit.

Backup: the four-share TI netlist

Four-share TI of $x \oplus (y \odot z)$ (Bilgin et al. 2015); each input four-shared, sixteen cross products $y_i \odot z_j$:

$$F_1 = x_2 \oplus \bigoplus_{i,j \in \{2,3,4\}} y_i z_j, \quad F_2 = x_3 \oplus y_1 z_3 \oplus y_1 z_4 \oplus y_3 z_1 \oplus y_4 z_1 \oplus y_1 z_1,$$
$$F_3 = x_4 \oplus y_1 z_2 \oplus y_2 z_1, \qquad F_4 = x_1.$$

A *first-order* threshold implementation — never claimed beyond order one. Coppula certifies orders 1 and 2 and finds the order-3 witness: a counterexample on a gadget with a *known* failure order, not a flaw.

Barthe, Gilles et al. (2015). **"Verified Proofs of Higher-Order Masking".** In: *Advances in Cryptology – EUROCRYPT 2015*. Vol. 9056. Lecture Notes in Computer Science. Springer, pp. 457–485. doi: 10.1007/978-3-662-46800-5_18.

Barthe, Gilles et al. (2017). **"Coupling Proofs are Probabilistic Product Programs".** In: *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. ACM, pp. 161–174. doi: 10.1145/3009837.3009896.

Barthe, Gilles et al. (2019). **"maskVerif: Automated Verification of Higher-Order Masking in Presence of Physical Defaults".** In: *Computer Security – ESORICS 2019*. Ed. by Kazue Sako, Steve A. Schneider, and Peter Y. A. Ryan. Vol. 11735. Lecture Notes in Computer Science. Cham: Springer International Publishing, pp. 300–318. isbn: 978-3-030-29959-0. doi: 10.1007/978-3-030-29959-0_15.

Bilgin, Begül et al. (2015). **"Threshold Implementations of Small S-boxes".** In: *Cryptography and Communications* 7.1, pp. 3–33. doi: 10.1007/s12095-014-0104-7.

Bloem, Roderick et al. (2018). **"Formal Verification of Masked Hardware Implementations in the Presence of Glitches".** In: *Advances in Cryptology – EUROCRYPT 2018*. Vol. 10821. Lecture Notes in Computer Science. Springer, pp. 321–353. doi: 10.1007/978-3-319-78375-8_11.

- Chari, Suresh et al. (1999). **"Towards Sound Approaches to Counteract Power-Analysis Attacks".** In: *Advances in Cryptology – CRYPTO '99*. Vol. 1666. Lecture Notes in Computer Science. Springer, pp. 398–412. doi: 10.1007/3-540-48405-1_26.
- Duc, Alexandre, Stefan Dziembowski, and Sebastian Faust (2014). ***Unifying Leakage Models: from Probing Attacks to Noisy Leakage.*** Cryptology ePrint Archive, Paper 2014/079. url: https://eprint.iacr.org/2014/079.
- Fenner, Stephen A., Lance J. Fortnow, and Stuart A. Kurtz (1994). **"Gap-Definable Counting Classes".** In: *Journal of Computer and System Sciences* 48.1, pp. 116–148. doi: 10.1016/S0022-0000(05)80024-8.
- Groß, Hannes, Stefan Mangard, and Thomas Korak (2016). **"Domain-Oriented Masking: Compact Masked Hardware Implementations with Arbitrary Protection Order".** In: *Proceedings of the 2016 ACM Workshop on Theory of Implementation Security (TIS@CCS)*. ACM, p. 3. doi: 10.1145/2996366.2996426.
- Ishai, Yuval, Amit Sahai, and David Wagner (2003). **"Private Circuits: Securing Hardware against Probing Attacks".** In: *Advances in Cryptology - CRYPTO 2003*. Ed. by Dan Boneh. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 463–481. isbn: 978-3-540-45146-4.

- Knichel, David, Pascal Sasdrich, and Amir Moradi (2020). **"SILVER – Statistical Independence and Leakage Verification".** In: *Advances in Cryptology – ASIACRYPT 2020*. Vol. 12491. Lecture Notes in Computer Science. Springer, pp. 787–816. doi: 10.1007/978-3-030-64837-4_26.
- Kocher, Paul, Joshua Jaffe, and Benjamin Jun (1999). **"Differential Power Analysis".** In: *Advances in Cryptology – CRYPTO '99*. Vol. 1666. Lecture Notes in Computer Science. Springer, pp. 388–397. doi: 10.1007/3-540-48405-1_25.
- Valiant, Leslie G. (1979). **"The Complexity of Computing the Permanent".** In: *Theoretical Computer Science* 8.2, pp. 189–201. doi: 10.1016/0304-3975(79)90044-6.
- Wagner, Klaus W. (1986). **"The Complexity of Combinatorial Problems with Succinct Input Representation".** In: *Acta Informatica* 23.3, pp. 325–356. doi: 10.1007/BF00289117.